```
        std::packaged_task<result_type()> task(f);
        std::future<result_type> res(task.get_future());
        if(local_work_queue)                                    ❹
        {
            local_work_queue->push(std::move(task));
        }
        else
        {
            pool_work_queue.push(std::move(task));              ❺
        }
        return res;
    }
    void run_pending_task()
    {
        function_wrapper task;
        if(local_work_queue && !local_work_queue->empty())      ❻
        {
            task=std::move(local_work_queue->front());
            local_work_queue->pop();
            task();
        }
        else if(pool_work_queue.try_pop(task))                  ❼
        {
            task();
        }
        else
        {
            std::this_thread::yield();
        }
    }
    // rest as before
};
```

We've used a `std::unique_ptr<>` to hold the thread-local work queue ❷ because we don't want non-pool threads to have one; this is initialized in the `worker_thread()` function before the processing loop ❸. The destructor of `std::unique_ptr<>` will ensure that the work queue is destroyed when the thread exits.

`submit()` then checks to see if the current thread has a work queue ❹. If it does, it's a pool thread, and you can put the task on the local queue; otherwise, you need to put the task on the pool queue as before ❺.